Přednášky

Sbalit vše

- Návrhové vzory
- Dokumentace
- Přednáška 1
- Přednáška 2
- Přednáška 3
- Přednáška 4
- Přednáška 5
- Přednáška 6

ZalohaPrednasek

Cvičení 1

Seznámení s průběhem předmětu **Základy počítačové grafiky**.

- Předpokladem je znalost programování v jazyce C++ na úrovni třetího ročníku bakalářského studia (práce s projektem, debugování zdrojového kódu, objektové programování atd.).
- Znalost středoškolské matematiky a matematiky probírané na vysoké škole v předcházejících ročnících (přímka, funkce kosinus, operace s maticemi atd.).

Lineární algebra (Skripta: linearni_algebra.pdf).

- Cvičení jsou povinná a budou průběžně kontrolována.

Cílem předmětu je probrat a prakticky ověřit základní části počítačové grafiky (OpenGL) a jejich implementaci. Jednotlivé části budeme pro přehlednost přidávat do jednoho projektu a tvořit tak naši výslednou aplikaci.

Postupně budeme probírat:

1. Více modelů, shaderů
2. Načítání modelů (FBX)
3. Transformace
4. Kamera a promítání
5. Osvětlovací modely (Lambert, Phong, Blinn)
6. Normálová matice
7. Světla (bodové, směrové, reflektor). více světel
8. Textury, skybox, skydome
9. Identifikace modelů, unproject
10. Pohyb po úsečce, Bézierova kubika, Bézierova křivka po obloucích

Řiďte se pokyny vyučujícího na svém cvičení. Celkem lze získat **45 bodů** za zápočet. Přibližně to odpovídá **5 bodů** za každý splněný úkol. Součástí odevzdávání je i kontrola znalosti, které souvisí s danou částí.

První termín pro písemnou část bude v zápočtovém týdnu **7.12.2026** na poslední přednášce (kdy máme všichni volno). Další termín písemky probehne ve zkouškovém (přibližně v druhém týdnu zkouškového).

Mimo přednášky a cvičení existuje pro OpenGL spousta dostupné literatury a ukázek. Jako příklad uveďme learnopengl.com i s online knihou Joey de Vries: Learn OpenGL, ISBN: 978-90-90-33256-7 nebo online https://open.gl/ a kniha Alexander Overvoorde: Modern OpenGL Guide.
Vše podstatné si budeme vysvětlovat postupně na jednotlivých přednáškách a následně prakticky zkoušet na cvičeních.
Pro studium teorie lze využít také skripta Matematické základy počítačové grafiky.

Na cvičeních budeme používat **C++** (Visual Studio 2026) a další vybrané knihovny. Odzkoušejte dnes minimálně GLFW a GLM (později GLEW, Assimp, OpenCV, ...).

Cvičení jsou **povinná** a každou úlohu budeme odesílat přes kelvin.cs.vsb.cz, termínem odevzdání bude vždy neděle.
Cílem předmětu je vytvořit jednoduchý vykreslovací řetězec, kdy se budeme pohybovat ve 3D prostředí a budeme mít možnost určité interakce s objekty (C++,

OpenGL).
Výslednou aplikaci (Standartní zobrazovací řetězec) budeme průběžně vytvářet a konzultovat na cvičeních.
Pokud byste chtěli vyvíjet aplikaci v jiném jazyce, je to po domluvě možné.

```
#include <iostream>

int main()
{
 std::cout << "Hello World!\n";
 system("PAUSE");
 return 0;
}
```

Testovací příklad [main.cpp] //cvičení 1

Abychom si ušetřili práci, budeme využívat některé knihovny, které nám budou umožňovat rozšířit stávající možosti C/C++.

Knihovny (popřípadě jejich části) lze rozdělit podle způsobu použití na:

- Statickou část knihovny – pro kterou platí, že to co program z knihoven využívá, je vloženo přímo do spustitelného kódu (zvětšují velikost).
- Dynamická část knihovny – které se načítají za běhu. Tyto knihovny se přikládají k spustitelnému souboru samostatně a mohou být sdíleny mezi více programy.

Ve Windows mají statické knihovny obvykle příponu .lib (static library) a dynamické knihovny mají příponu .dll (dynamic linked library).

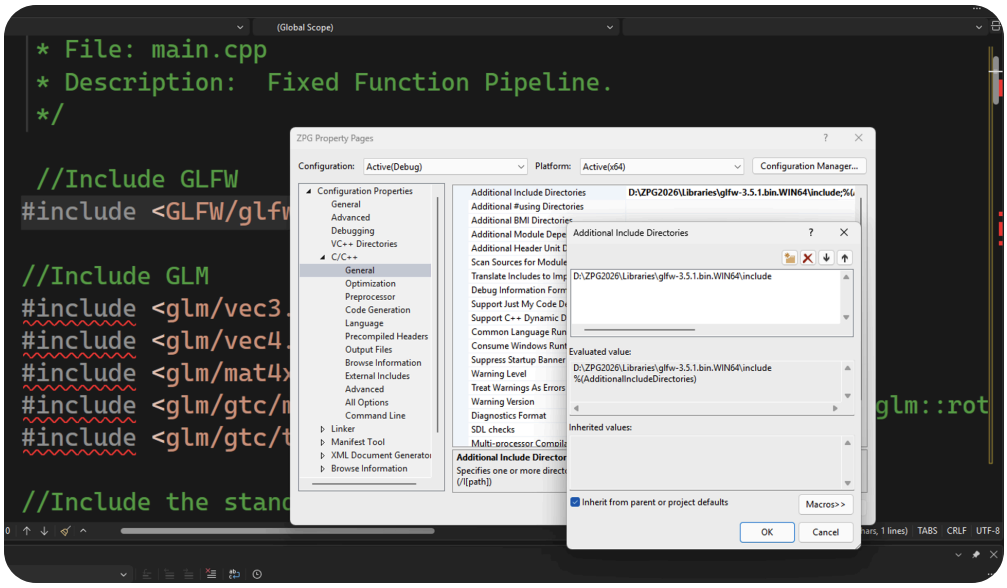
GLFW: http://www.glfw.org/download.html
GLM: https://glm.g-truc.net
GLAD: https://gen.glad.sh/ (GLEW)

---

opengl32.lib
glu32.lib
glfw3.lib

//assimp.lib, openCV.lib, glew32.lib ...

I na 64bitových systémech se používá soubor s názvem **opengl32.lib** a **glu32.lib**, který se přizpůsobuje automaticky dané architektuře (32bitové nebo 64bitové).



GLFW (http://www.glfw.org/)

GLFW is an Open Source, multi-platform library for creating windows with OpenGL contexts and receiving input and events. It is easy to integrate into existing applications and does not lay claim to the main loop.

GLM (http://glm.g-truc.net/) OpenGL Mathematics

OpenGL Mathematics (GLM) is a header only C++ mathematics library for graphics software based on the OpenGL Shading Language (GLSL) specifications. GLM provides classes and functions designed and implemented with the same naming conventions and functionalities than GLSL so that anyone who knows GLSL, can use GLM as well in C++.

GLAD (https://gen.glad.sh/) Multi-Language Vulkan/GL/GLES/EGL/GLX/WGL Loader-Generator

GLAD (Glad 2) is an open-source, multi-language loader-generator used in graphics programming to fetch official specifications and generate custom function loaders for OpenGL, Vulkan, EGL, GLES, GLX, and WGL.

Assimp (http://assimp.sourceforge.net/) Open Asset Import Library

Assimp is a portable Open Source library to import various well-known 3D model formats in a uniform manner. The most recent version also knows how to export 3d files and is therefore suitable as a general-purpose 3D model converter. See the feature list.

Dear ImGui (often called just **ImGui**) **(https://github.com/ocornut/imgui)** is a fast, portable, and self-contained graphical user interface library for C++ that uses an "immediate mode" design.

OpenCV (http://opencv.org/) Open Source Computer Vision Library

OpenCV is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products.

AntTweakBar (http://anttweakbar.sourceforge.net)

AntTweakBar is a small and easy-to-use C/C++ library that allows programmers to quickly add a light and intuitive graphical user interface into graphic applications based on OpenGL (compatibility and core profiles), DirectX 9, DirectX 10 or DirectX 11 to interactively tweak parameters on-screen.

FreeGLUT (http://freeglut.sourceforge.net/) Free OpenGL Utility Toolkit

Free-software/open-source alternative to the OpenGL Utility Toolkit (GLUT) library.

Relativní vs. absolutní cesta

D:\ZPG\Cviceni\Libraries\glew\lib\Release\Win32

vs.

..\..\Libraries\glew\lib\Release\Win32

$(ProjectDir)\Libraries\glm-master

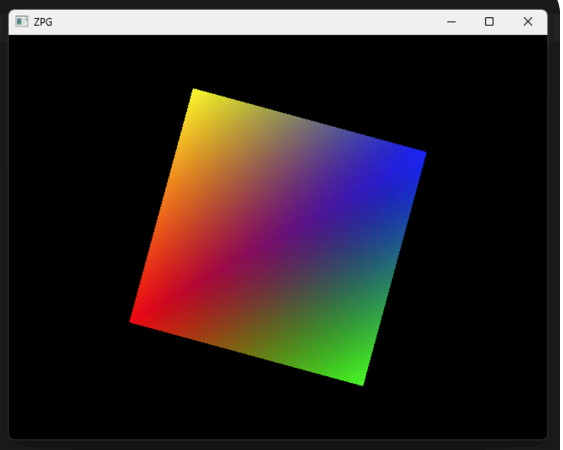
$(solutiondir)\Libraries

```
glBegin(GL_TRIANGLES);

glColor3f(1.f, 0.f, 0.f);
glVertex3f(-0.6f, -0.6f, 0.f);
glColor3f(0.f, 1.f, 0.f);
glVertex3f(0.6f, -0.6f, 0.f);
glColor3f(0.f, 0.f, 1.f);
glVertex3f(0.6f, 0.6f, 0.f);

glColor3f(1.f, 0.f, 0.f);
glVertex3f(-0.6f, -0.6f, 0.f);
glColor3f(1.f, 1.f, 0.f);
glVertex3f(-0.6f, 0.6f, 0.f);
glColor3f(0.f, 0.f, 1.f);
glVertex3f(0.6f, 0.6f, 0.f);

glEnd();
```



Úkoly k procvičování

1. Vytvořte z aktuálního trojúhelníku čtverec, jehož čtvrtý vrchol bude žlutý.
2. Upravte aplikaci tak, aby se při stisku klávesy změnil plynulý směr rotace.
3. Změňte si osu rotace.
4. Zkuste rotovat kolem obecného bodu (např. kolem jednoho z rohů čtverce).
5. Zprovozněte si knihovny GLM a GLAD, budeme je potřebovat příští týden.
6. Používejte verzovací systém, můžete použít gitlab.vsb.cz.
7. Zopakujte si debugování (procházení, krokování, vypisování proměnných atd.) zdrojového kódu.

main.cpp

Cvičení 2

Toto cvičení budeme věnovat úpravě zdrojového kódu a přechod na OpenGL 3.3+ (přechod na moderní OpenGL).

Pro pohodlné používání příkazu OpenGL, budeme potřebovat knihovnu GLAD, která která se postará o **načtení funkcí OpenGL** dostupných v ovladači grafické karty.

- Program vytvoří OpenGL kontext.

- Grafický ovladač poskytne adresy funkcí, které podporuje.
- Tyto adresy je potřeba v programu načíst.
- GLAD tento proces udělá za nás.

GLAD (OpenGL Loader-Generator) gen.glad.sh umožňuje vygenerovat knihovnu pro OpenGL verze 4.6 (lze i nižší) a můžeme vygenerovat pouze hlavičkové soubory (v opačném případě budeme mít *.*h* a *.c soubory a musíme je přidat do projektu.

```
//Include GLAD
//Only define this in one file
#define GLAD_GL_IMPLEMENTATION
#include <glad/gl.h>

//Include GLFW
#include <GLFW/glfw3.h>

//Include the standard C++ headers
#include <stdlib.h>
#include <stdio.h>
#include <fstream>
#include <string>
#include <iterator>
#include <iostream>
#include <vector>
```

Upravíme začátek main funkce, abychom inicializovali OpenGL a zjistili aktuální verzi.

```
// Pointer to the GLFW window
GLFWwindow* window;

// Initialize GLFW
if (!glfwInit())
	exit(EXIT_FAILURE);

//Initialization of a specific version
/*
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
glfwWindowHint(GLFW_OPENGL_PROFILE,
GLFW_OPENGL_CORE_PROFILE);  //*/


window = glfwCreateWindow(800, 600, "ZPG", NULL, NULL);
if (!window)
{
	glfwTerminate();
	exit(EXIT_FAILURE);
}

glfwMakeContextCurrent(window);
```

```
glfwSwapInterval(1);

// Initialize GLAD and load OpenGL function pointers
if (!gladLoadGL((GLADloadfunc)glfwGetProcAddress))
{
	printf("GLAD initialization failed\n");
	return -1;
}

// Get version info
printf("OpenGL Version: %s\n", glGetString(GL_VERSION));
printf("Vendor %s\n", glGetString(GL_VENDOR));
printf("Renderer %s\n", glGetString(GL_RENDERER));
printf("GLSL %s\n", glGetString(GL_SHADING_LANGUAGE_VERSION));
int major, minor, revision;
glfwGetVersion(&major, &minor, &revision);
printf("Using GLFW %i.%i.%i\n", major, minor, revision);
```

Model - kombinace **VBO + VAO**

VBO - Vertex Buffer Object - oblast paměti na grafické kartě, do které nahrajeme data vrcholů modelu.

VAO - Vertex Array Object - návod na to, jak máme data interpretovat.

```
float points[] = {
   0.0f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f,
   0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f,
  -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 1.0f
};

	//vertex buffer object (VBO)
	GLuint VBO = 0;
	glGenBuffers(1, &VBO); // generate the VBO
	glBindBuffer(GL_ARRAY_BUFFER, VBO);
	glBufferData(GL_ARRAY_BUFFER, sizeof(points), points, GL_STATIC_DRAW);

	//Vertex Array Object (VAO)
	GLuint VAO = 0;
	glGenVertexArrays(1, &VAO); //generate the VAO
	glBindVertexArray(VAO); //bind the VAO
	glEnableVertexAttribArray(0); //enable vertex attributes
	glEnableVertexAttribArray(1);
	glBindBuffer(GL_ARRAY_BUFFER, VBO);
	// index, number of components, data type, normalized, vertex stride, offset
	glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (GLvoid*)0);
	glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (GLvoid*)(3 * sizeof(float)));
```

Aktuálně se náš model (pole points) skládá z **3*float** souřadnic pro pozici a **3*float** souřadnic pro barvu. Za chvilku budeme ale potřebovat místo barvy hlavně normály (normálové vektory v jednotlivých vrcholech), které budeme využívat pro výpočet osvětlení.

Načtěte a zkompilujte ze souboru Vertex a Fragment shader a následně vytvořte Shader program.

```
GLuint createShaderFromFile(GLenum shaderType, const char* shaderFile)
{
	// Creates an empty shader
	GLuint shaderID = glCreateShader(shaderType);

	if (shaderID == 0)
	{
		std::cout << "Unable to create shader" << std::endl;
		exit(EXIT_FAILURE);
	}

	//Loading the contents of a file into a variable
	std::ifstream file(shaderFile);
	if (!file.is_open())
	{
		std::cout << "Unable to open file " << shaderFile << std::endl;
		glDeleteShader(shaderID);
		exit(-1);
	}
	std::string shaderCode((std::istreambuf_iterator<char>(file)), std::istreambuf_iterator<char>());

	// Set the shader source code
	const char* source = shaderCode.c_str();
	glShaderSource(shaderID, 1, &source, nullptr);

	// Compile the shader source code
	glCompileShader(shaderID);

	// Check specialization/compilation status
	GLint success;
	glGetShaderiv(shaderID, GL_COMPILE_STATUS, &success);
	if (!success)
	{
		char infoLog[1024];
		glGetShaderInfoLog(shaderID, sizeof(infoLog), nullptr, infoLog);
		std::cout
			<< "Shader failed:\n"
			<< infoLog << std::endl;
		glDeleteShader(shaderID);
		exit(1);
	}
	return shaderID;
}

// Create and compile the vertex and fragment shaders
GLuint vertexShader = createShaderFromFile(GL_VERTEX_SHADER, "shaders/basic.vert");
GLuint fragmentShader =	createShaderFromFile(GL_FRAGMENT_SHADER, "shaders/basic.frag");

//Create and link the shader program
GLuint shaderProgram = glCreateProgram();
```

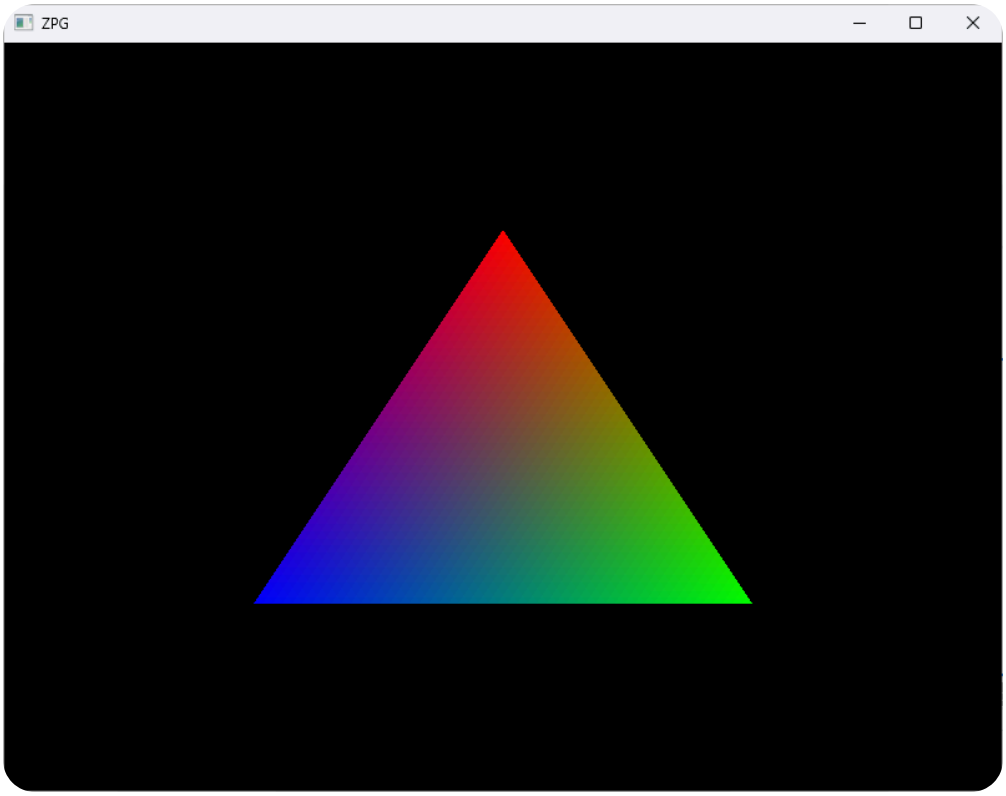
```
glAttachShader(shaderProgram, fragmentShader);
glAttachShader(shaderProgram, vertexShader);
glLinkProgram(shaderProgram);
```

Uvnitř vykreslovací smyčky while postupně nastavte konkrétní shader, VAO a vykreslete trojúhelníky.

```
while (!glfwWindowShouldClose(window))
{
	// Clear color and depth buffer
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	glUseProgram(shaderProgram);
	glBindVertexArray(VAO);

	// Draw a triangles
	glDrawArrays(GL_TRIANGLES, 0, 3); //mode,first,count

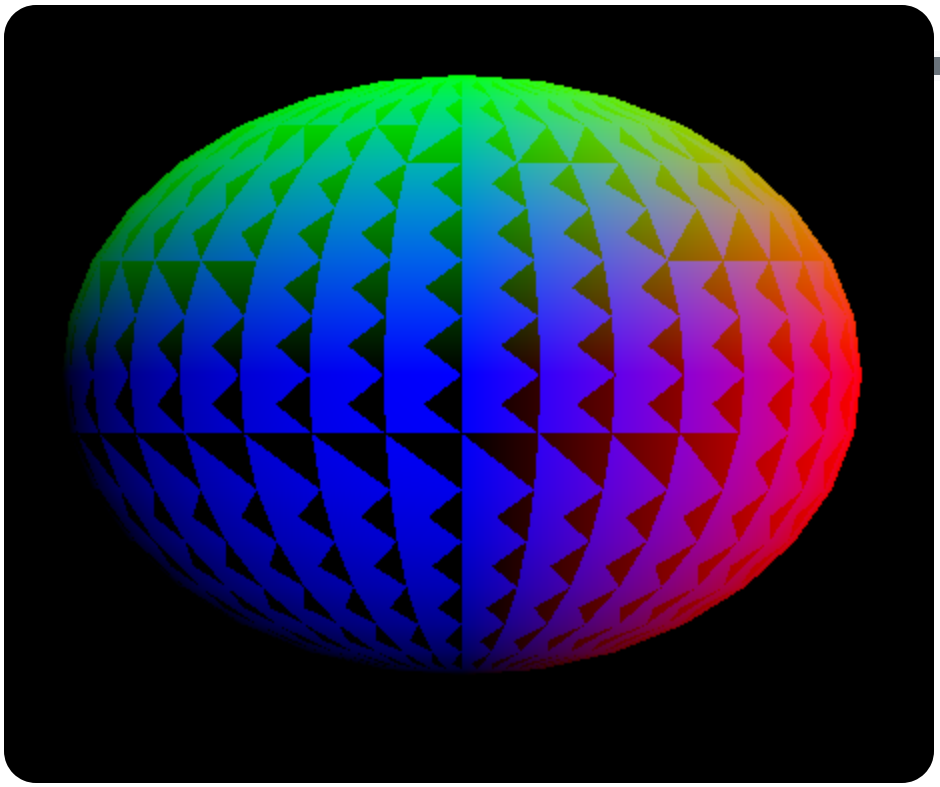
	// Display the rendered frame and process events
	glfwSwapBuffers(window);
	glfwPollEvents();
}
```



Pro správné zobrazení ploch je potřeba zapnout z-buffer (více na některé z dalších přednáškách).

```
glEnable(GL_DEPTH_TEST);//Do depth comparisons and update the depth buf
while (!glfwWindowShouldClose(window))
```

{



//K čemu asi slouží tohle?
gl_Position = vec4(0.4*position+vec3(-0.5,0.0,0.0), 1.0);

Zdrojové kódy si vhodně upravte na objektový kód (třídy, metody). Při návrhu OOP se řiďte věcmi, které jsme si řekli na přednášce a cvičeních.

Image preview

Autor: Pitek Daniel

Příklad souboru **main.cpp**

```
/**
 * @file main.cpp
 *
 * @brief Main function
 *
 * @author ...
  **/
#include "Application.h"
int main(void)
```

```
{
        Application* app = new Application();
        app->initialization(); //OpenGL inicialization

        //Loading scene
        app->createShaders();
        app->createModels();
        app->run(); //Rendering
}
```

Výsledné zdrojové kódy (CPP a H) odešlete podle termínů na https://kelvin.cs.vsb.cz/.

Úkoly na cvičení

1. Přidejte si do projektu knihovnu GLAD, abychom mohli používat programovatelný vykreslovací řetězec.
2. Udělejte v shaderu schválně chybu a ověřte, že se aplikace nepustí a že vypíše chybu.
3. Zkuste si vytvořit více modelů a vykreslovat je, vytvořte si jednoduchá primitiva, jako trojúhelník, čtverec, krychle atd. (příště již přidáme složitější modely).
4. Vyzkoušejte si vykreslit více modelů pomocí více shader programu (shader programy se můžou lišit třeba výslednou barvou ve fragment shaderu).
5. Upravte pozici ve vertex shaderu tak, že ji přenásobíte číslem (např. 0,5) nebo k pozici přičtěte vektor (+vec3(0.1, -0.3, 0.7)).
6. Zaměřte se i na objektový kód podle domluvy na přednášce a cvičení. Upravte jej tak, aby jste si zpřehlednili funkčnost a každá třída podle svého názvu odpovídala za přidělenou část a funkcionalitu (aplikace, shadery, modely atd.).

glad.zip

basic.frag

basic.vert

models.zip

Cvičení 3

Protože nám odpadla přednáška, tak se zatím omezíme pouze na znalosti středoškolské matematiky a více budeme řešit příští týden po přednášce.

Mezi základní transformace patří

- posun (translace)
- změna měřítka (scale)
- otáčení (rotace)

Vyzkoušejte si na cvičení všechny tři transformace (posun a změnu měřítka jsme již zkoušeli minulý týden)

Vyjdeme z rotace v rovině, kdy rotaci bodu A=[x,y] kolem středu souřadné soustavy (počátku) S=[0,0] o úhel α proti směru hodinových ručiček získáme bod A'=[x',y'], který spočteme jako:

```
x' =  x*cos(α) - y*sin(α)
y' =  x*sin(α) + y*cos(α)
```

Rotace bodu A=[x,y,z] v prostoru kolem **osy y** vypočteme jako (bližší vysvětlení na přednášce):

```
x' =  cos(α) * x + sin(α) * z
y' =  y
z' = -sin(α) * x + cos(α) * z
```

Ověřte si funkčnost všech tří transformací na cvičeních a využijte možnosti uniformních proměnných v shaderu a vytvořte ovládání pomocí kláves. Nepoužívejte matice ale výše popsané rovnice.

Posílání proměnné

Vytvoříme si v shader programu uniformní proměnnou, kterou budeme následně nastavovat.

```
#version 330 core
    uniform vec3 fragmentColor = vec3(1, 1, 0);
void main()
{
    ...
```

Pro nastavení proměnné v shaderu musíme nejdříve použít funkci glGetUniformLocation(), která vrátí location (číselný identifikátor) uniform proměnné v shader programu. Nezapomeňte přidat test na -1, kterou shader program vrátí pokud proměnnou nenašel.

```
int varLocation = glGetUniformLocation(shaderProgram, "fragmentColor")
```

Pošlete vektor s barvou do shader programu

```
glUseProgram(shaderProgram);
int varLocation = glGetUniformLocation(shaderProgram, "fragmentColor");
glUniform3f(varLocation, 1.0f, 0.0f, 0.0f);
glUseProgram(0);
```

K čemu a proč je dobré používat příkaz ***glUseProgram(0)***?

glUniformMatrix4fv(idModelTransform, 1, GL_FALSE, &M[0][0]);

//location, count, transpose, *value

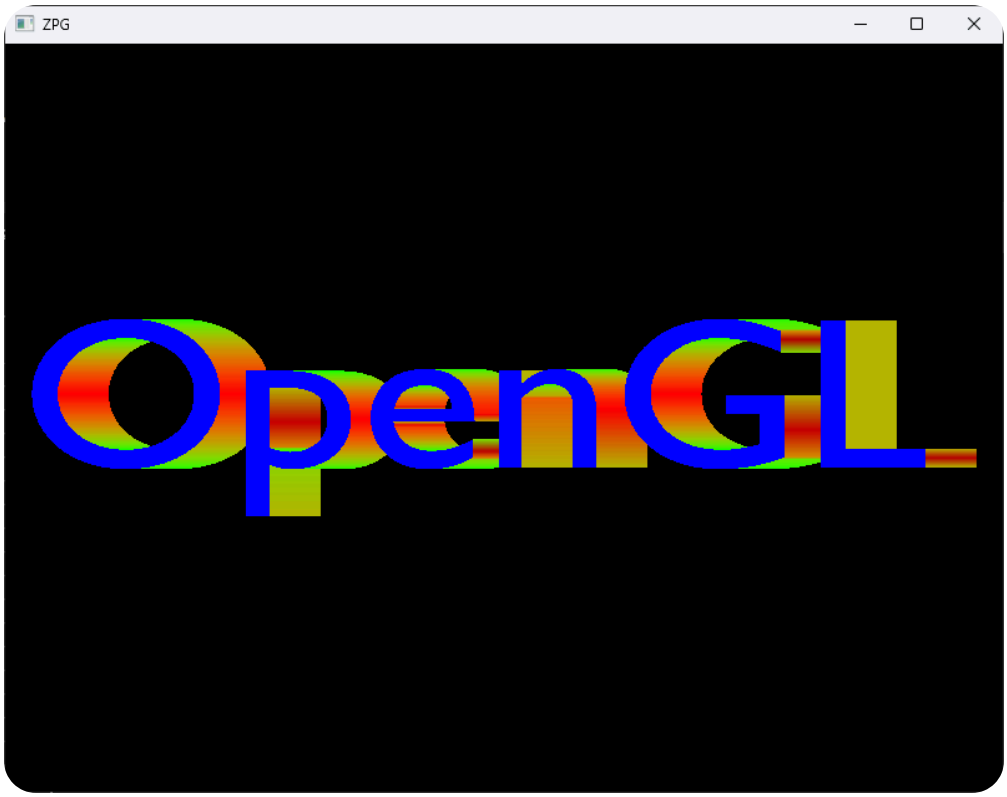
Příště budeme potřebovat:

```
M = glm::mat4(1.0f);
GLint matrixLocation = glGetUniformLocation(shaderProgram, "modelMatrix");
glUniformMatrix4fv(matrixLocation, 1, GL_FALSE, glm::value_ptr(M));

M = glm::lookAt(glm::vec3(5.0f, 0.0f, 0.0f), glm::vec3(0.f, 0.f, 0.f), glm::vec3(0.f, 1.f, 0.f));
matrixLocation = glGetUniformLocation(shaderProgram, "viewMatrix");
glUniformMatrix4fv(matrixLocation, 1, GL_FALSE, glm::value_ptr(M));

M = glm::perspective(glm::radians(45.0f), 800.f / 600.f, 0.1f, 100.0f);
matrixLocation = glGetUniformLocation(shaderProgram, "projectMatrix");
glUniformMatrix4fv(matrixLocation, 1, GL_FALSE, glm::value_ptr(M));
```

Vytvořte si 3D model vašeho loginu a vložte ho do scény. Login si můžete vymodelovat sami nebo využít AI pro vygenerování.



Upravte si vhodně zdrojový kód a přidejte třídu **Scene**, která bude reprezentovat scénu a bude možné je za běhu přepínat.

- Ve zdrojovém kódu je povoleno používat pouze angličtinu, a to včetně komentářů. Použití nevhodných nebo vulgárních výrazů bude hodnoceno jako neodevzdaný úkol.
- Pokud používáte cizí soubory (např. moje modely v hlavičkových souborech), nesmíte do nich zasahovat ani je upravovat.
- Pokud některou část programu nevytvoříte sami (např. ji vygeneruje AI), musí být tato skutečnost řádně označena. V opačném případě může být práce považována za plagiátorství.
- Vlastní zdrojové soubory (např. soubory obsahující vlastní třídy) by měly obsahovat hlavičku se jménem a loginem autora.

Úkoly na cvičení

1. V případě kuličky, kdy normály zobrazujeme jako barvu, použijte na výslednou barvu absolutní hodnotou.
2. Prakticky si otestujte základní transformace (přidejte si ovládání pomocí kláves).
3. Vytvořte si vlastní model loginu (můžete použít AI pro vygenerování) a použijte ho ve scéně.

4. Upravte si třídy a metody podle domluvy na přednášce a cvičeních.

OpenGL.h

Login pomocí AI

Cvičení 4

Transformace (Projektivní prostor)

Vytvoříme si matici 4x4 (transformaci) pomocí knihovny GLM (popřípadě sami). Vyzkoušíme základní transformace (translace, rotace, scale).

```
glm::mat4 M = glm::mat4(1.0f); // construct identity matrix

M = glm::rotate(glm::mat4(1.0f), angle, glm::vec3(0.0f, 1.0f, 0.0f));
M = glm::rotate(M, angle, glm::vec3(1.0f, 0.0f, 0.0f));
M = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, 0.0f, 0.5f));
M = glm::scale(glm::mat4(1.0f), glm::vec3(0.5f));
```

Upravíme vertex shader tak, že si uvnitř vytvoříme uniformní proměnnou, kam budeme naši matici posílat. A následně každý vrchol touto proměnnou (matici) vynásobíme ve vertex shaderu. Pozor na správné násobení (nasobení matic není komutativní).

```
//Vertex shader
...
uniform mat4 modelMatrix;
...
//main function
main{
...
```

Přenásobíme každý vrchol ve vertex shaderu naší transformační matici (už víme proč je mat4 a proč vec4()).

```
gl_Position = modelMatrix * vec4(position, 1.0);
```

Zjistíme pozici naši uniformní proměnné v shader programu (přidejte test na -1, kterou vrátí pokud proměnnou nenašel)
Pošlete naši matici do shaderu

```
//Render

glUseProgram(shaderProgram);
GLint modelMatrixLocation = glGetUniformLocation(shaderProgram, "modelMatrix");
//glUniformMatrix4fv(location, count, transpose, value)
glUniformMatrix4fv(modelMatrixLocation,1,GL_FALSE,glm::value_ptr(M));
glUseProgram(0);
```

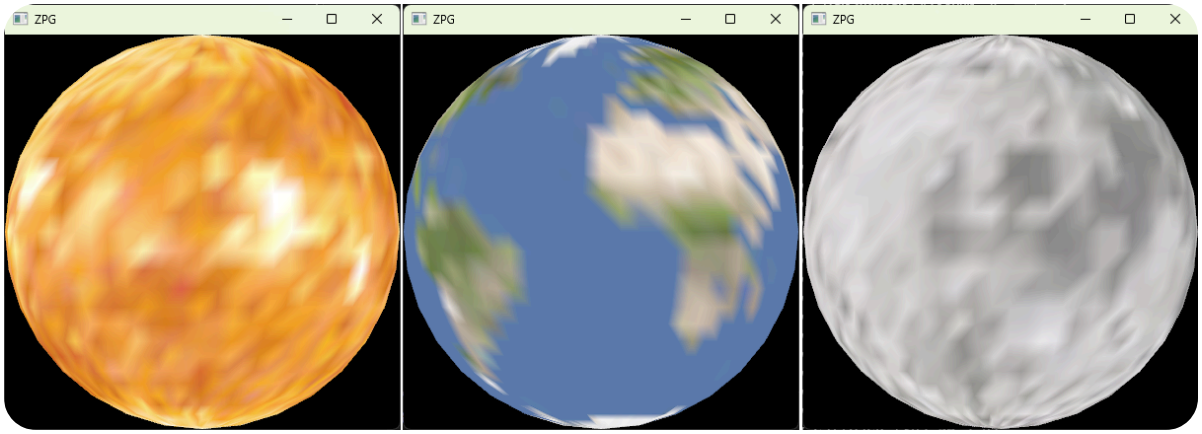
Příště budeme potřebovat:

```
M = glm::lookAt(glm::vec3(5.0f, 0.0f, 0.0f), glm::vec3(0.f, 0.f, 0.f), glm::vec3(0.f, 1.f, 0.f));
matrixLocation = glGetUniformLocation(shaderProgram, "viewMatrix");
glUniformMatrix4fv(matrixLocation, 1, GL_FALSE, glm::value_ptr(M));

M = glm::perspective(glm::radians(45.0f), 800.f / 600.f, 0.1f, 100.0f);
matrixLocation = glGetUniformLocation(shaderProgram, "projectMatrix");
glUniformMatrix4fv(matrixLocation, 1, GL_FALSE, glm::value_ptr(M));
```

Přidejte si další modely, použijte příklad z minulého cvičení, kdy si naimportujete model do Blenderu a skriptem si vytvoříte vlastní model ve vhodném formátu.

Příklad použití AI při generování modelu planet. Skript v Blenderu převedl z modelu s texturou barvu jednotlivých vrcholů z textury přímo na vrcholy. Výsledek při použití barev u vrcholů je sice po grafické stránce horší, ale později přidáme i samotnou texturu a vše bude vypadat lépe. Modely Slunce, Země a Měsíce máte přiložené u cvičení. Můžete si přidat další modely.



Vytvořil jsem Discord server, na který můžete nahrát své vlastní výsledky, aby si je mohli prohlédnout i ostatní. Můžete tak své výsledky porovnat s ostatními studenty. https://discord.gg/NMvNZaRmgN

---

1. Vytvořte si vlastní model na kterém ověříte možnosti skládání transformací.

2. Ověřte si osy a souřadný systém v OpenGL (osy **x**, **y** a **z**).
3. Na transformace můžete použít návrhový vzor kompozit (nakreslete a vysvětlete).
4. Měli byste umět vysvětlit, jak fungují transformace v OpenGL, co se přesně děje a proč používáme projektivní prostor.
5. Vytvořte si model pohybující se sluneční soustavy (slunce, zemi, měsíc, ostatní planety apod.).
6. Vhodně upravte OOP (refaktoring kódu), dodržujte názvy a vztahy podle domluvy na přednášce (vlastní názvy znemožňují práci v teamu).

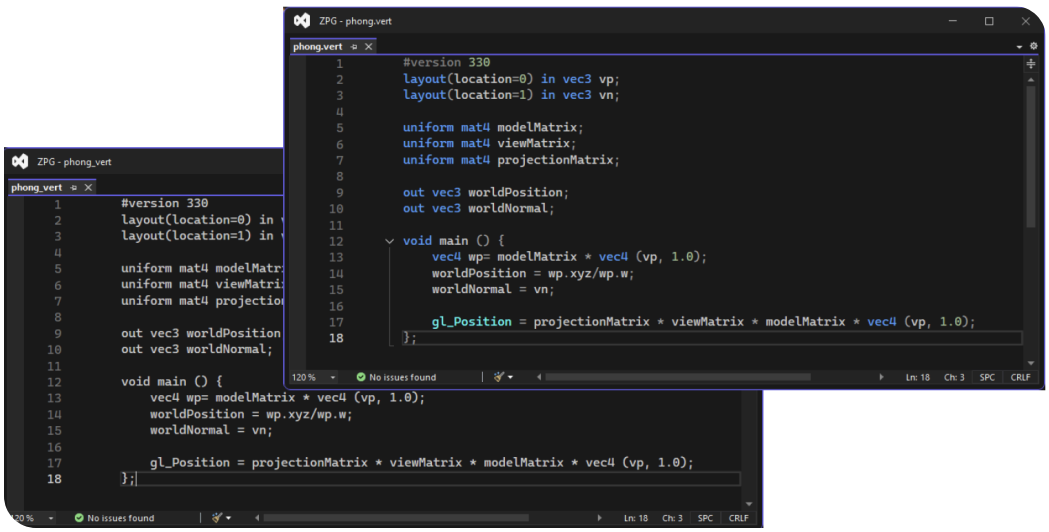
earth.h

sun.h

moon.h

Cvičení 5

1. Pro lepší přehlednost můžete ve VS použít např. **GLSL language integration (for VS2022)** (funguje i ve VS2026). Autor Daniel Scherzer [http] odkaz na GIT (Github releases).



2. Vytvořte třídu `Camera`, která bude reprezentovat kameru ve scéně. Třída bude obsahovat pohledovou matici (`viewMatrix`) a projekční matici (`projectionMatrix`).

3. Rozšiřte vertex shader o uniformní proměnné pro modelovou, pohledovou a projekční matici. Pro začátek můžete pohledovou a projekční matici nastavit na jednotkovou matici.

```
uniform mat4 modelMatrix = mat4(1.0);;
uniform mat4 viewMatrix = mat4(1.0);;
uniform mat4 projectionMatrix = mat4(1.0);;

void main()
{
        gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec
}
```

4. Pro pohledovou matici můžete použít funkci `glm::LookAt(eye, look, up);` (viz přednáška).

```
glm::mat4 Camera::getCamera(void) {
        return glm::lookAt(eye, eye + target, up);
}
```

```
5. Pro výpočet vektoru pohledu kamery můžete použít (viz přednáška). Ne
```

```
        target.x = sin(alpha) * cos(fi);
        target.z = sin(alpha) * sin(fi);
        target.y = cos(alpha);
```

```
6. Nastavte projekční matici na perspektivní promítání, vyzkoušet si mů
   poměr okna by jste měli upravovat podle rozlišení okna, ve kterém se
   Pozor: z_near nastavte na hodnotu 0.1, nepoúživejte hodnotu 0.
```

```
        //Projection matrix
        //60° Field of View, 4:3 ratio, display range : 0.1 unit <-> 10
        glm::mat4 projection = glm::perspective(glm::radians(60.0f), 4.
```

```
7. Zaměřte se opět na OOP (Podívejte se a když tak můžete použít věci j
```

Křížový odkaz, v případě kamery se nám bude hodit, aby kamera pracovala s shader programem a shader program mohl pracovat s kamerou.
Dopředná deklarace (Forward declaration) odstraňuje problémy z důvodu křížového odkazu.

```
//------- ShaderProgram.h ------
class Camera; // dopredna deklarace (z duvodu krizoveho odkazu)
class ShaderProgram{
  private:
    Camera* m_camera;
```

```
  public:
    Shader(Camera* camera);
    void metoda();
};

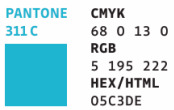

//------- ShaderProgram.cpp ------
#include "ShaderProgram.h"
#include "Camera.h"
ShaderProgram::ShaderProgram(){
  ...
}

//------- Camera.h ------
class ShaderProgram; // dopredna deklarace (z duvodu krizoveho odkazu)
class Camera{
  private:
    ShaderProgram* m_shaderProgram;
  public:
    Camera(ShaderProgram* shaderProgram);
    void metoda();
};

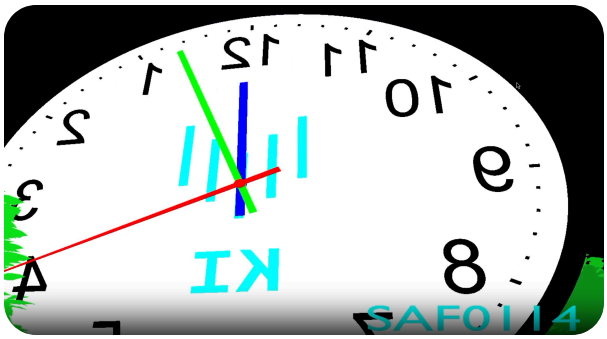
//------- Camera.cpp ------
#include "Camera.h"
#include "ShaderProgram.h"
Camera::Camera(){
  ...
}
```



Barva loga



Pokud se vám model (je v lokálním souřadném stytému, který nemusí být totožný s vaším souřadným systémem) nezobrazí správně, tak si jej upravte. Zde stačí použít scale(-1,1,1).



`gl_VertexID` v GLSL je vestavěná proměnná, která ti říká ID vrcholu, který se právě zpracovává ve vertex shaderu. Můžeme ji využít při vykreslování našeho stromu z předcházejících cvičení, kdy rozdělíme kmen s větvemi a listí.

```
void main()
{
   if(gl_VertexID>36443)   color = vec3(0.f, 1.f, 0.f);
   else color = vec3(1.f, 0.f, 0.f);
   gl_Position = modelMatrix * vec4(position, 1.0);
}
```



Pro obarvení listů nebyl použitý osvětlovací model, ale Intenzita barvy byla lineárně namapována na interval $\langle 0.3, 1 \rangle$ v závislosti na absolutní hodnotě y složky normálového vektoru povrchu. Je možné si to libovolně změnit a nastavit.

```
float shade = 0.3 + 0.7 * abs(normal.y);
FragColor = vec4(shade * color, 1.0);
```

Vytvořil jsem Discord server, na který můžete nahrát své vlastní výsledky, aby si je mohli prohlédnout i ostatní. Můžete tak své výsledky porovnat s ostatními studenty.
https://discord.gg/NMvNZaRmgN

Vytvořte si kameru (View a Projection matici).

Procházejte les (na přednášce jsme se domluvili, že les má min. 74 stromů).

Pro propojení třídy Camera a Shader můžete využijte návrhový vzor Observer (proč?).

Vytvořte si hodiny z přiloženého modelu (použijte jden drawCall).

clock.h

clock.zip

clock.blend

Cvičení 6

Zapnutí z-bufferu (princip na přednášce)

```
glEnable(GL_DEPTH_TEST);
```

Vytvořte si různé shadery podle přednášky (ukázka shaderu)

```
#version 460 core

layout (location = 0) in vec3 position;
layout (location = 1) in vec3 color;
layout (location = 2) in vec3 normal;

uniform mat4 modelMatrix=mat4(1.0);
uniform mat4 viewMatrix=mat4(1.0);
uniform mat4 projectionMatrix=mat4(1.0);

out vec3 worldPosition=vec3(0.0);
out vec3 worldNormal=vec3(0.0,1.0,0.0);

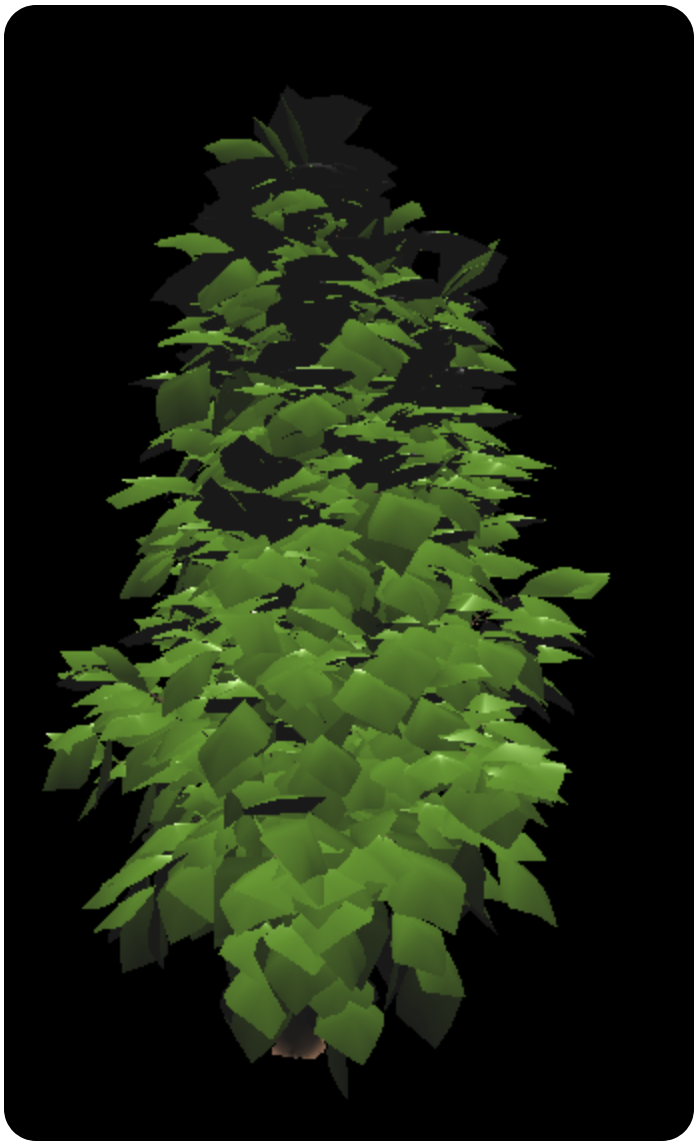

void main(void){
    vec4 pos = modelMatrix * vec4(position,1.0f);
    worldPosition = pos.xyz/pos.w;
    mat3 normalMatrix = mat3(transpose(inverse(modelMatrix)));
    worldNormal = normalize(normalMatrix * normal);
    gl_Position = (projectionMatrix * viewMatrix * modelMatrix) * vec4(
}
```

```
//Fragment shader
in vec4 worldPosition;
in vec3 worldNormal;

out vec4 out_Color;
void main(void){
  vec3 lightPosition= vec3(10.0,10.0,10.0); //Point Light position
```

```
  ...
  float dotProduct = max(dot(normalize(lightToVector), normalize(worldNormal)),
0.0);
  vec4 diffuse = dotProduct * vec4( 0.385, 0.647, 0.812, 1.0);
  vec4 ambient = vec4( 0.1, 0.1, 0.1, 1.0);
  out_Color = ambient + diffuse;
}
```

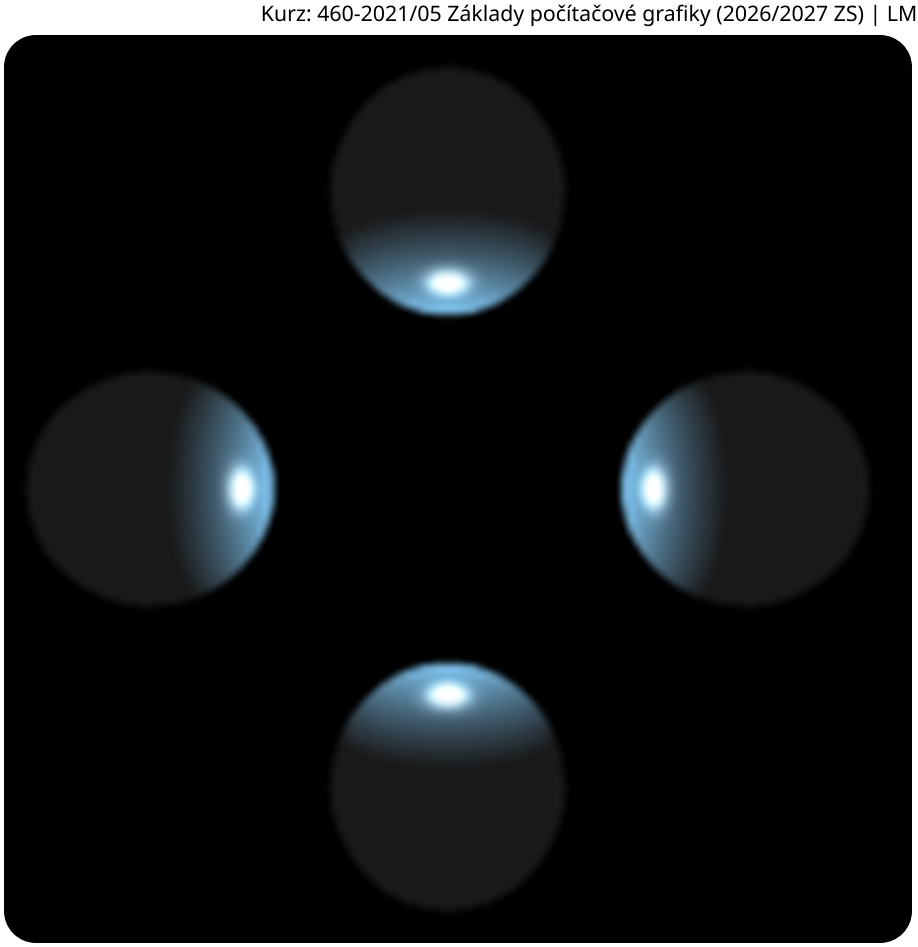
GLSL language integration (for VS2022)} Daniel Scherzer [http]



Testovací scéna 1

Světlo v [0,0,0], čtyři kulové plochy souměrně rozmístěny a kamera nad objekty.

Normálovou matici budeme řešit příští přednášku.



Testovací scéna 2 – Sluneční soustava

Připravte si scénu pro vizualizaci naší sluneční soustavy (Slunce, Země a Měsíc) a jejich simulaci (Země obíhá kolem Slunce, Měsíc obíhá kolem Země), oběžnými dráhami mohou být kružnice. Jak jistě tušíte, budou zde důležité správné transformace. Upravte si velikosti a poměry tak, aby byla simulace dobře viditelná. Využijte osvětlovací model. Pro Slunce použijte konstantní shader, aby bylo rovnoměrně osvětlené po celém povrchu. U ostatních planet použijte Phongův a Blinnův osvětlovací model, aby byly osvětlené ze strany přivrácené ke Slunci.

Zatím nepočítáme s vrženým stínem, takže ani když se Země dostane mezi Slunce a Měsíc nebo Měsíc mezi Slunce a Zemi, žádný stín tam nebude.

FOV (Field of View)
Nastavte FOV na vhodnou hodnotu a vyzkoušejte různé úhly, například 30°, 45°, 60°, 90°, 120°, 150° a 170°. Sledujte, jak se mění výsledný pohled na scénu. Jak FOV ovlivňuje scénu?

Cvičení 7 Zvýrazněno

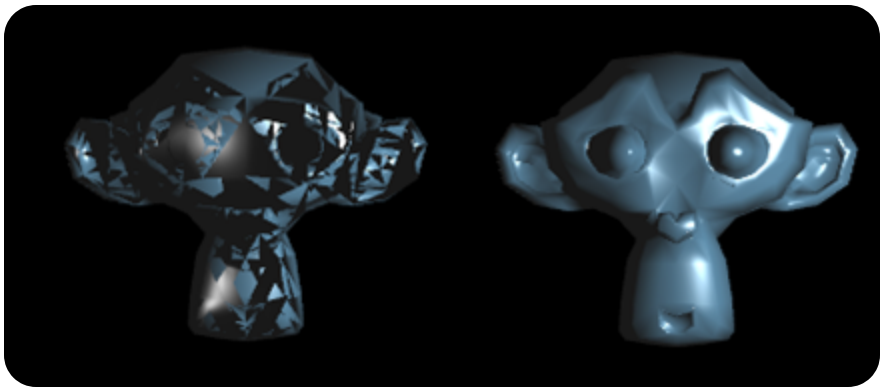
1. Přidání normálové matice

Normálová matice (převádí normály z lokálníhou souřadného systému do globálního), více na přednášce.

```
mat3 normalMatrix=transpose(inverse(mat3(modelMatrix)));
vec3 normalWorld=normalMatrix * vertexNormal;
```

Jak vypočtu odražený paprsek pro výpočet zrcadlové složky?
reflect https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/reflect.xhtml

2. Řešení viditelnosti (z-buffer)



glEnable(GL_DEPTH_TEST); // nastavit až po inicializaci OpenGL (např. před smyčku while).

//třídění ploch podle vzdálenosti (více v dalších přednáškách)

3. Útlum světla

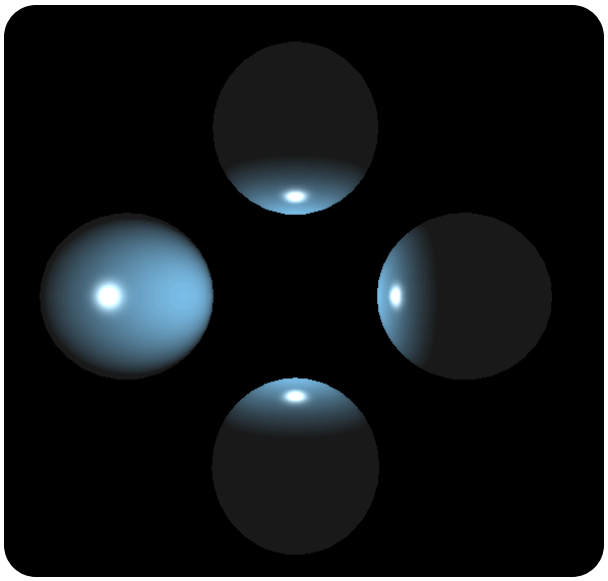
Hodnoty nejsou obvykle předem pevně dané, upravujeme je podle požadavků na výslednou scénu. Vyzkoušejte nastavení různých typů útlumu.

```
// Point light attenuation calculation
float attenuation(float d, float c, float l, float q)
{
  // d = distance from the light to the fragment
  // c = constant attenuation factor
  // l = linear attenuation factor
  // q = quadratic attenuation factor
```

```
  float att=1.0/ (c + l*d + q*d*d)
  return clamp(att , 0.0 , 1.0);
}
float att = attenuation(dist, 1.0, 0.09, 0.032);
```

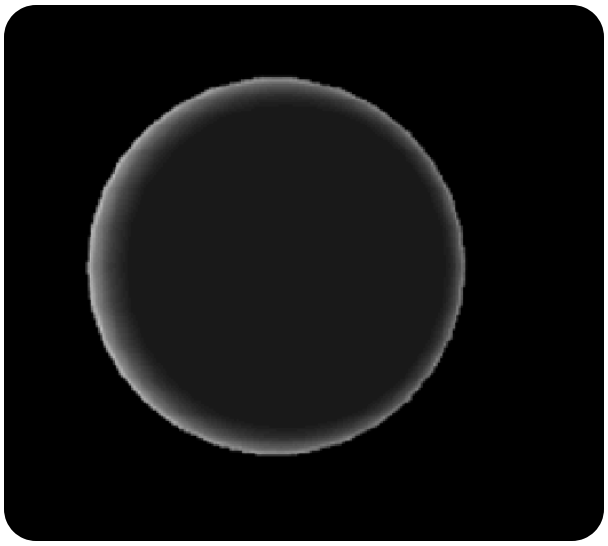
Testovací úkoly popsané na přednášce

1. Zkontrolujte a ověřte si prakticky transformace normál (bez transformací, s modelovou matici a s normálovou matici). Vysvětlete rozdíl, odvození a funkcionalitu.

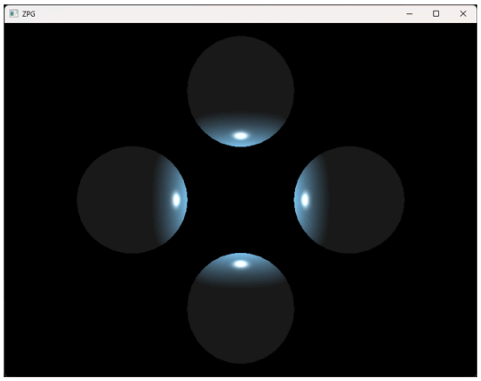


Co se stalo a proč?

2. Ověřte a opravte následující chybu v shaderu. Proč se děje? Kdy se to stane?

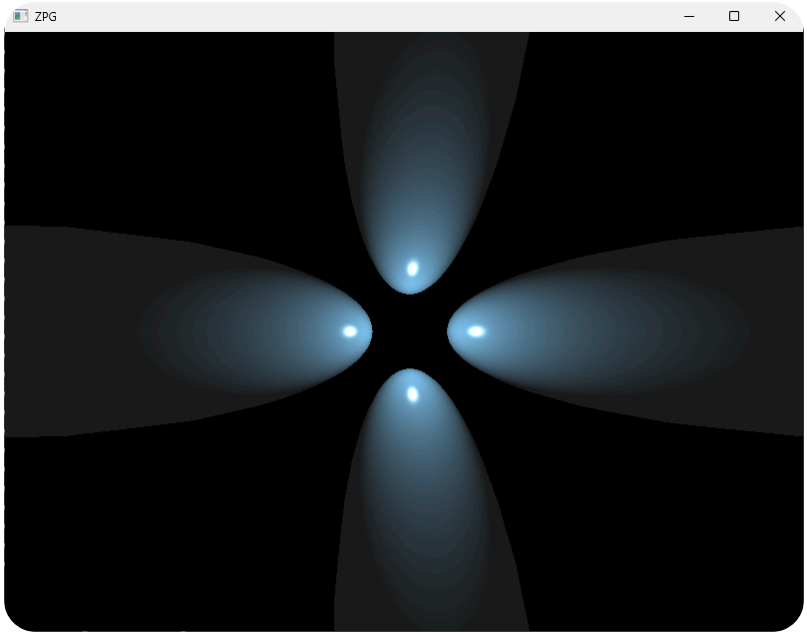


3. Otestujte správné chování při změně okna (glViewport a perspective).





4. Co se tady stalo a proč?



5. Vyzkoušejte následující matici. Proč ji testujeme? Co má ověřit? Kde ji máte použít?

$$modelMatrix = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 5 \end{bmatrix}$$

```
glm::mat4 modelMatrix = glm::mat4(
        1.0f, 0.0f, 0.0f, 0.0f,
        0.0f, 1.0f, 0.0f, 0.0f,
        0.0f, 0.0f, 1.0f, 0.0f,
        0.0f, 0.0f, 0.0f, 5.0f
);
```

6. Proveďte kontrolu zdrojového kódu a refaktorujte jeho části.

© 2012 - 2026 VŠB-TUO

Kontaktovat technickou podporu

Běží na technologii Moodle Pty Ltd